Q1. Application: Cybersecurity Threat Analysis System

A cybersecurity system analyzes threats using the recurrence T(n)-T(n-1)+nlogn. Apply the substitution method to solve the recurrence. Expand step-by-step and derive the complexity. Analyze how the algorithm scales with increasing threat data. Interpret efficiency using asymptotic notation.

Aim

To solve the recurrence relation **T(n) = T(n − 1) + n log n** using the **substitution method**, derive its time complexity, and analyze the algorithm's scalability in cybersecurity threat analysis.

Algorithm

1. Start with the recurrence relation.
2. Expand the recurrence repeatedly using substitution.
3. Continue until the base case $T(1)$.
4. Express the recurrence as a summation.
5. Evaluate the summation.
6. Derive the asymptotic time complexity.
7. Analyze the scalability using asymptotic notation.

Source code:

```python
import math

# Function to solve T(n) = T(n-1) + n log n
def recurrence(n):
    if n == 1:
        return 1
    return recurrence(n - 1) + n * math.log2(n)

# Main Program
n = int(input("Enter the value of n: "))

result = recurrence(n)

print("\nRecurrence Value T(", n, ") =", round(result, 2))
print("Time Complexity : O(n^2 log n)")
print("Space Complexity: O(n)")
```

Step-by-Step Analysis

Given,

$$T(n) = T(n-1) + n\log n$$

Expand:

$$= T(n-2) + (n-1)\log(n-1) + n\log n$$
$$= T(n-3) + (n-2)\log(n-2) + (n-1)\log(n-1) + n\log n$$

Continuing,

$$= T(1) + \sum_{k=2}^{n} k\log k$$

Since,

$$\sum_{k=1}^{n} k\log k = \Theta(n^2\log n)$$

Therefore,

$$\boxed{T(n) = \Theta(n^2\log n)}$$

Sample input and output

```
Enter the value of n: 5

Recurrence Value T( 5 ) = 27.36
Time Complexity : O(n^2 log n)
Space Complexity: O(n)

=== Code Execution Successful ===
```

Result

The recurrence relation **T(n)=T(n−1)+n log n** was solved using the substitution method. The overall time complexity is **Θ(n² log n)** and the space complexity is **O(n)**. The algorithm's running time increases quadratically with an additional logarithmic factor as the volume of threat data grows.

Q2. Application: Data Backup Optimization System

A backup system uses an algorithm with recurrence T(n) = 2T(n/2) + √n. Apply the Master Theorem to determine the time complexity. Compare f(n) with n^log_b a and identify the correct case. Analyze performance for large datasets. Interpret efficiency

Aim

To determine the time complexity of the recurrence relation **T(n) = 2T(n/2) + √n** using the Master Theorem and analyze its efficiency for large backup datasets.

Algorithm

1. Identify the values of **a**, **b**, and **f(n)**.
2. Compute $n^{\log_b a}$.
3. Compare $f(n)$with $n^{\log_b a}$.
4. Identify the applicable Master Theorem case.
5. Derive the time complexity.
6. Analyze the algorithm's efficiency for large datasets.

Source Code

```
import math

def backup_time(n):
    if n == 1:
        return 1
    return 2 * backup_time(n // 2) + math.sqrt(n)

n = int(input("Enter the value of n: "))

result = backup_time(n)

print("T(", n, ") =", round(result, 2))
print("Time Complexity : O(n)")
print("Space Complexity: O(log n)")
```

Sample Input and Output

```
Enter the value of n: 8
T( 8 ) = 20.49
Time Complexity : O(n)
Space Complexity: O(log n)
```

Analysis

The recurrence relation is:

$$T(n) = 2T\left(\frac{n}{2}\right) + \sqrt{n}$$

- The problem is divided into two equal subproblems at each recursive step.
- The additional work performed at each level is √n, which is smaller than the recursive work.
- Comparing $f(n) = \sqrt{n}$with $n^{\log_b a} = n$, we get:

$$\sqrt{n} = O(n^{1-\varepsilon}), \varepsilon = \frac{1}{2}$$

- Therefore, the recurrence satisfies Case 1 of the Master Theorem.
- The recursive calls dominate the running time, resulting in an overall complexity of:

$$\boxed{\boldsymbol{\Theta(n)}}$$

Result

Using the Master Theorem, the recurrence

$$T(n) = 2T\left(\frac{n}{2}\right) + \sqrt{n}$$

falls under Case 1, and the overall time complexity is:

$$\boxed{\Theta(n)}$$

Thus, the data backup optimization system is efficient and scales linearly as the amount of backup data increases.